

UNIVERSIDAD DE COSTA RICA

PF-3311 · Agentes Virtuales Inteligentes

Entregable 2: Avance de Agente e Investigación

Documento de avance de la Prueba de Concepto

Agente conversacional con andamiaje socrático para programación
introductoria

Estudiante	Hansell Solís Ramírez
Curso	PF-3311 Agentes Virtuales Inteligentes
Ciclo	I Ciclo, 2026
Repositorio	agente-andamiaje-ucr
Documento	Avance actual del agente, diseño metodológico y plan de evaluación

Nota para entrega: Antes de generar la versión final en PDF, sustituir el enlace del video de demostración y verificar que el README del repositorio contenga instrucciones reproducibles de ejecución y que no se suban llaves API ni archivos .env.

Resumen ejecutivo

Este documento presenta el avance actual del agente conversacional con andamiaje socrático para programación introductoria. **La Prueba de Concepto ya demuestra integración funcional entre una interfaz web, un motor de decisión pedagógica, un LLM base y un sistema de registro de eventos en JSON.** El avance más importante respecto al Entregable 1 es que el proyecto dejó de estar únicamente en fase de diseño y pasó a una implementación capaz de sostener sesiones por participante, conversar con el estudiante, modular el nivel de andamiaje y registrar evidencia conductual para evaluación posterior.

La siguiente evolución técnica consiste en agregar autenticación, persistencia en base de datos y un modelo de estudiante basado en evidencia observada. El objetivo no es reemplazar el motor actual de conversación ni la arquitectura de tres capas, sino conservarlos y envolverlos con una capa de memoria persistente que permita recuperar conversaciones anteriores, conceptos demostrados, niveles de andamiaje, sticky count histórico y métricas acumuladas.

Criterio de alcance: El documento se alinea con la consigna del Entregable 2: resume cambios desde el Entregable 1, describe el estado actual del agente, presenta el diseño metodológico, define métricas e instrumentos, y propone protocolos de interacción para evaluación piloto o experta.

1. Resumen de cambios desde el Entregable 1

1.1 Feedback recibido y aprendizaje metodológico

El Entregable 1 definió el proyecto como un agente conversacional con embodiment visual y andamiaje pedagógico adaptativo para apoyar el pensamiento computacional en programación introductoria. A partir de la retroalimentación del curso, de la preparación metodológica y de los resultados exploratorios obtenidos con el agente textual, el foco del Entregable 2 se concentró en demostrar la viabilidad técnica del núcleo conversacional antes de expandir el componente multimodal o embodied.

- Se priorizó validar primero el motor pedagógico de andamiaje socrático, porque es el componente que determina si el agente guía sin sustituir el razonamiento del estudiante.
- Se incorporó la necesidad de instrumentar el sistema con logs reproducibles, no solo con impresiones cualitativas sobre si el prompt “funcionó”.
- Se identificó la necesidad de separar la fricción pedagógica deseada de la fricción técnica producida por fallas del proveedor LLM o por falta de persistencia.
- Se decidió mantener una arquitectura modular de tres capas para poder evolucionar el sistema sin reescribir el motor principal.

1.2 Cambios realizados a la propuesta original

Elemento	Propuesta Entregable 1	Avance actual	Justificación
Problema	Riesgo de dependencia cognitiva por uso de IA generativa en programación	Se mantiene, pero se operacionaliza como problema de diseño del andamiaje: cuándo	Permite evaluar el agente con logs, niveles de ayuda y eventos observables.

	introdutoria.	guiar, cuándo aflojar y cómo evitar sustitución.	
RQs	Diseño de agente con embodiment, comparación texto/embodiment y patrones de interacción.	Se conserva la ruta de investigación, pero el avance se concentra en el núcleo textual con andamiaje y memoria persistente.	El núcleo conversacional debe ser viable antes de evaluar embodiment como capa adicional.
Agente	Tutor virtual con orientación socrática y presencia visual.	Tutor conversacional textual con niveles de andamiaje, sticky count, friction ceiling y detección de conceptos.	Permite probar el comportamiento pedagógico principal sin depender aún del avatar.
Arquitectura	Aplicación web con LLM, posible voz, persistencia y despliegue.	PoC web de tres capas con sesiones, chat LLM y logs JSON; transición planificada hacia base de datos y memoria persistente.	La persistencia permite continuidad, análisis longitudinal y perfil del estudiante.
Evaluación	Evaluación con expertos y piloto controlado.	Diseño metodológico con SUS, NASA-TLX, logs, fidelidad del prompt, focus group/entrevista y revisión experta.	Triangula percepción, carga, conducta y calidad pedagógica del prompt.

1.3 Decisiones principales

1. Conservar el motor de conversación y andamiaje actual, porque ya implementa la lógica central de guía socrática.
2. No modificar la arquitectura de tres capas; la persistencia y la autenticación se agregan como evolución alrededor del núcleo existente.
3. Pasar de logs JSON como único almacenamiento a una base de datos que mantenga compatibilidad con esos logs.
4. Representar el perfil del estudiante con estados cualitativos basados en evidencia observada: Dominado, En progreso y Requiere apoyo.
5. Mantener el enfoque socrático, pero hacerlo adaptativo mediante niveles, sticky count y friction ceiling.

2. Estado actual del agente

2.1 Descripción general del prototipo

La PoC actual es una aplicación web orientada a sesiones de programación introductoria. El estudiante interactúa con un agente conversacional que recibe preguntas, código parcial o errores, y

responde siguiendo una estrategia de andamiaje socrático. El sistema registra cada interacción como evento estructurado, lo que permite analizar posteriormente la trayectoria de ayuda, el progreso conceptual y los puntos de atasco.

- Arquitectura de tres capas: presentación, servicios/orquestación y datos/logs.
- Sesiones por participante mediante participantId y sessionId.
- Chat conversacional conectado con un LLM base.
- Registro de logs en JSON.
- Niveles de andamiaje 1, 2 y 3.
- Sticky Count para rastrear intentos sin progreso.
- Friction Ceiling para controlar cuándo aumentar la ayuda.
- Detección de conceptos demostrados durante la interacción.
- Registro de prompts, respuestas del agente y tiempos de respuesta.

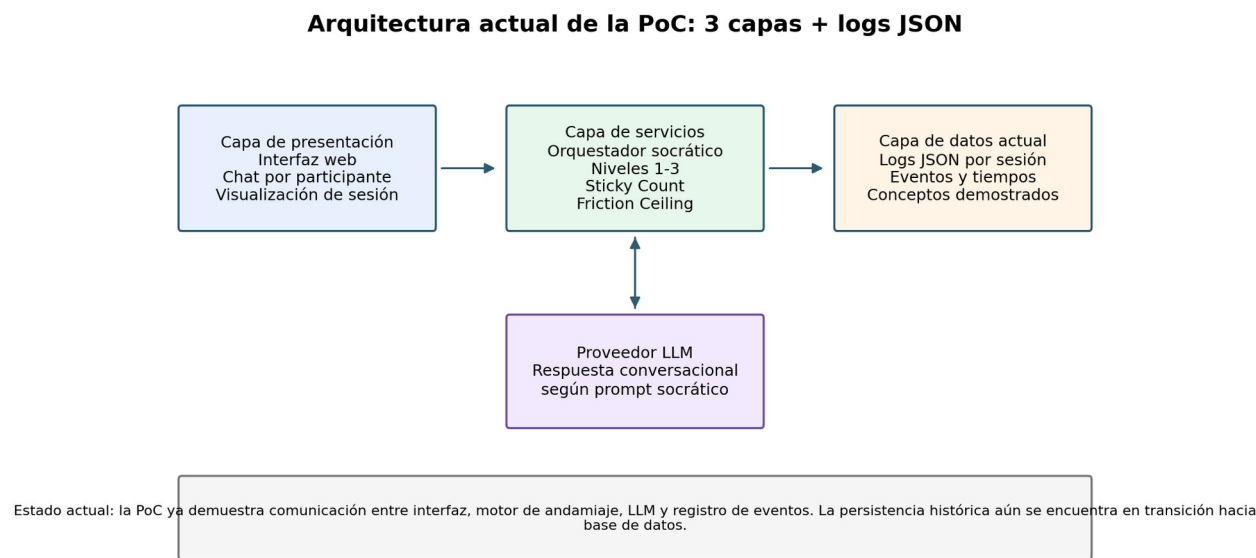


Figura 1. Arquitectura actual de la PoC. Muestra la integración mínima ya demostrada: interfaz web, orquestador socrático, LLM y logs JSON.

2.2 Información registrada actualmente en los logs

Campo	Descripción	Uso metodológico
participantId	Identificador del participante.	Permite agrupar sesiones e interacciones por estudiante.
sessionId	Identificador único de la sesión.	Permite reconstruir una sesión completa.
scaffoldLevel	Nivel de andamiaje aplicado.	Permite analizar si el agente escala la ayuda.
stickyCount	Conteo de interacciones sin avance suficiente.	Indica atasco o necesidad de aumentar apoyo.
conceptosDemostrados	Conceptos observados en la interacción.	Base para construir el perfil persistente del estudiante.
AI_PROMPT	Mensaje del estudiante.	Permite codificar dudas, código,

		errores y peticiones.
AI_RESPONSE	Respuesta del agente.	Permite evaluar fidelidad del prompt y calidad de ayuda.
responseTimeMs	Tiempo de respuesta del LLM.	Permite medir latencia y fricción técnica.

2.3 Funcionalidades implementadas vs. pendientes

Componente	Estado	Evidencia/alcance	Pendiente inmediato
Interfaz web de chat	Implementado	Permite interacción estudiante-agente en una sesión.	Mejorar visualización del perfil del estudiante.
Conexión con LLM	Implementado	El agente responde a prompts reales del usuario.	Robustecer reintentos ante errores del proveedor.
Sesiones por participante	Implementado	Cada interacción se agrupa por participantId y sessionId.	Vincular sesión a usuario autenticado.
Logs JSON	Implementado	Registra prompts, respuestas, niveles, sticky count y tiempos.	Migrar a base de datos sin perder compatibilidad.
Niveles de andamiaje	Implementado	Niveles 1, 2 y 3 disponibles en el motor.	Visualizar trayectoria por sesión.
Sticky Count y Friction Ceiling	Implementado	Controlan señales de atasco y escalamiento.	Persistir histórico por estudiante.
Detección de conceptos	Implementado	Registra conceptos demostrados.	Convertir evidencia en estados de perfil.
Autenticación	Pendiente	Necesaria para memoria persistente.	Usuario, contraseña y bcrypt.
Base de datos	Pendiente	Necesaria para persistencia longitudinal.	Diseñar tablas y migrar JSON.
Perfil del estudiante	Diseñado / pendiente	Estados: Dominado, En progreso, Requiere apoyo.	Crear panel visual y reglas de actualización.

2.4 Evolución hacia memoria persistente

La evolución técnica propuesta no altera el motor principal del agente. La arquitectura de tres capas se mantiene, pero la capa de datos se fortalece con persistencia relacional, autenticación y un modelo de estudiante. Esta decisión permite conservar la lógica actual de interacción y, al mismo tiempo, habilitar continuidad entre sesiones.

Evolución propuesta: memoria persistente y perfil del estudiante

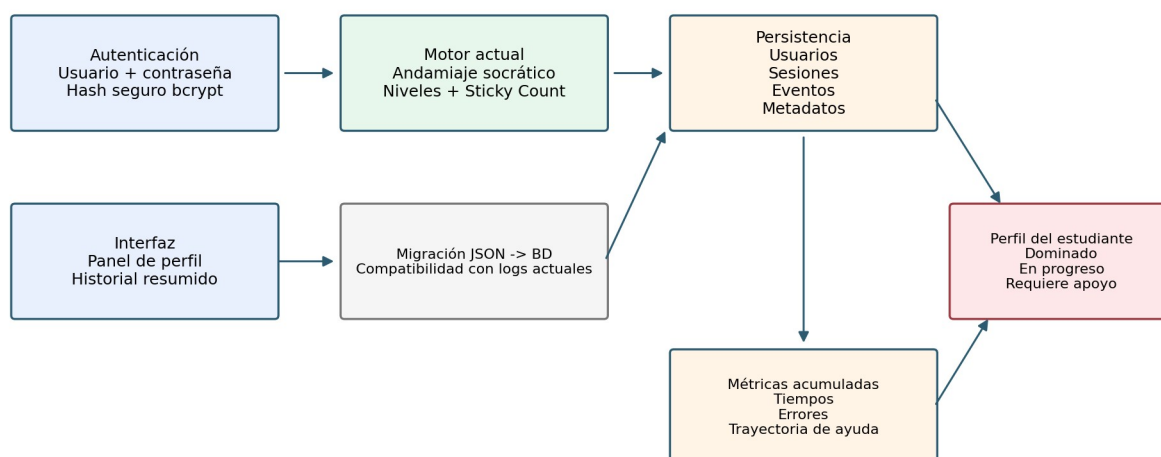


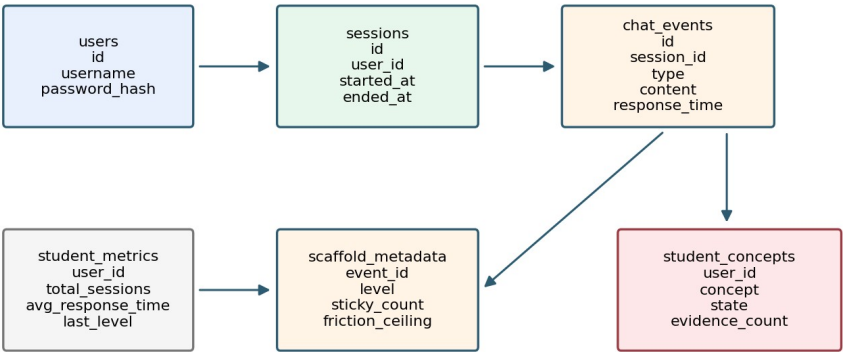
Figura 2. Evolución propuesta hacia autenticación, base de datos y perfil persistente del estudiante.

2.5 Modelo persistente del estudiante

El perfil del estudiante no se representará con porcentajes de conocimiento. Se utilizarán estados cualitativos basados en evidencia observada durante las interacciones. Esto evita presentar una falsa precisión numérica y permite justificar cada estado con eventos concretos registrados en logs.

Estado	Criterio operativo inicial	Ejemplo
Dominado	El estudiante aplica el concepto correctamente en más de una ocasión y puede explicarlo o verificarlo.	Variables -> Dominado
En progreso	El estudiante usa el concepto parcialmente, pero requiere pistas o comete errores corregibles.	Condicionales -> En progreso
Requiere apoyo	El estudiante muestra atasco reiterado, confusión conceptual o dependencia del agente.	Bucles -> Requiere apoyo

Modelo de datos propuesto para memoria persistente



Estados del perfil: Dominado, En progreso, Requiere apoyo. No se usan porcentajes de conocimiento; cada estado se actualiza con evidencia observada en eventos.

Figura 3. Modelo de datos propuesto. Las entidades nuevas se agregan alrededor de la sesión y los eventos, manteniendo compatibilidad con los logs existentes.

3. Diseño metodológico del estudio

3.1 Tipo de estudio

Se propone un estudio piloto de Prueba de Concepto complementado con evaluación experta. La finalidad no es generalizar estadísticamente a toda la población estudiantil, sino verificar la viabilidad técnica del sistema, la coherencia del andamiaje y la capacidad de los instrumentos para capturar usabilidad, carga y comportamiento de interacción.

Justificación ética y de alcance: En coherencia con la consigna del curso, la evaluación se plantea como piloto y/o revisión con expertos. Si se desea realizar una evaluación formal con usuarios reales para investigación publicable, debe gestionarse la autorización correspondiente del CEC antes de recolectar datos con personas participantes.

3.2 Participantes

Elemento	Diseño propuesto
Perfil	Personas con conocimiento básico de programación en Python o expertos docentes/asistentes con experiencia en programación introductoria.
Tamaño muestral piloto	3 a 5 expertos para revisión pedagógica y técnica; 5 a 10 participantes para piloto controlado si se cuenta con autorización y consentimiento.
Criterios de inclusión	Conocimiento mínimo de variables, condicionales, listas y ciclos; disponibilidad para realizar una

	sesión de aproximadamente 60 minutos.
Criterios de exclusión	Participantes sin consentimiento informado, sin disponibilidad para completar instrumentos post-tarea o que ya hayan participado en el diseño del prompt.
Reclutamiento	Invitación por conveniencia dentro del entorno académico del curso, priorizando revisión experta y piloto técnico de baja escala.
Protección de datos	Uso de identificadores pseudónimos; no almacenar contraseñas en texto plano; no subir API keys ni archivos .env al repositorio.

3.3 Procedimiento general

Flujo metodológico propuesto para el estudio de evaluación



El diseño combina métricas perceptuales (SUS, NASA-TLX), datos conductuales (logs) y revisión cualitativa/experta para evitar conclusiones basadas en una sola fuente.

Figura 4. Flujo metodológico propuesto para el estudio piloto y la evaluación experta.

1. Presentar el objetivo del estudio y solicitar consentimiento.
2. Asignar o crear un usuario de prueba en la PoC.
3. Recolectar información mínima de perfil: experiencia previa en Python, uso previo de IA y familiaridad con agentes conversacionales.
4. Solicitar la resolución de uno o más ejercicios de programación introductoria usando el agente.
5. Registrar automáticamente eventos de chat, niveles de andamiaje, sticky count, conceptos demostrados y tiempos de respuesta.
6. Aplicar instrumentos post-tarea: SUS, NASA-TLX y preguntas abiertas o entrevista breve.
7. Analizar la sesión triangulando percepción, carga, comportamiento y fidelidad del prompt.

4. Métricas

Las métricas se seleccionan para responder si el agente es usable, si su andamiaje produce una carga tolerable, si mantiene fidelidad al prompt y si los eventos observables permiten construir un perfil persistente del estudiante.

Métrica	Qué mide	Origen	Conexión con RQ	Momento
Usabilidad percibida	Facilidad de uso, aprendibilidad e intención de reuso.	SUS	Evalúa si el agente puede adoptarse como apoyo académico.	Después de la tarea.
Carga cognitiva	Demanda mental, temporal, esfuerzo, frustración y rendimiento percibido.	NASA-TLX Raw	Permite verificar si el andamiaje guía sin volver la interacción excesivamente costosa.	Después de la tarea.
Patrones de interacción	Prompts, turnos, duración, reintentos, errores y cambios de ejercicio.	Logs	Permite observar el comportamiento real, no solo percepción.	Durante la sesión.
Nivel de andamiaje	Escalamiento de ayuda 1-3.	scaffoldLevel	Permite evaluar si el agente adapta su ayuda al atasco.	Durante la sesión.
Sticky Count	Interacciones sin progreso suficiente.	stickyCount	Indica cuándo el estudiante requiere mayor apoyo.	Durante la sesión.
Conceptos demostrados	Evidencia de uso de conceptos de programación.	conceptosDemostrados	Base del perfil persistente del estudiante.	Durante y después.
Fidelidad del prompt	Cumplimiento de reglas: no dar solución, no evaluar, mantener rol.	Análisis de respuestas del agente	Mide si el comportamiento real se ajusta al diseño pedagógico.	Post-sesión.
Robustez técnica	Errores del proveedor, latencia y fallas de sesión.	Eventos de error y responseTimeMs	Distingue fricción técnica de fricción pedagógica.	Durante la sesión.

5. Instrumentos de recolección de datos

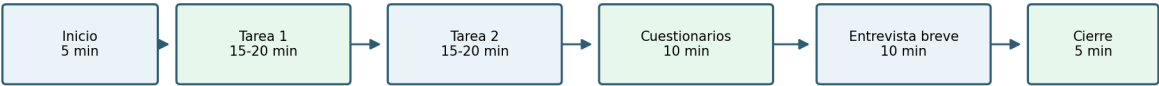
Instrumento	Base / literatura	Qué mide	Administración	Momento
System Usability Scale (SUS)	Brooke (1996); Sauro y Lewis (2016).	Usabilidad percibida global del agente.	Cuestionario de 10 ítems en escala Likert 1-5.	Inmediatamente después de la interacción.
NASA-TLX Raw	Hart y Staveland (1988).	Carga de trabajo subjetiva: mental, física, temporal, rendimiento, esfuerzo y frustración.	Formulario post-tarea en escala 0-100.	Inmediatamente después de la interacción.
Logs del sistema	Análisis	Eventos, turnos,	Registro	Durante toda la

	conductual de interacción humano-IA.	tiempos, errores, niveles de andamiaje y conceptos.	automático en JSON y, en la evolución, base de datos.	sesión.
Análisis de fidelidad del prompt	Guardrails y evaluación de tutores LLM; Liffiton et al. (2023).	Cumplimiento de reglas pedagógicas del agente.	Codificación de respuestas del agente contra reglas del system prompt.	Post-sesión.
Entrevista breve / focus group	Métodos cualitativos de HCI y evaluación de experiencia de usuario.	Percepción, frustración, utilidad, claridad y necesidades de rediseño.	Guion semiestructurado breve.	Después de cuestionarios.
Revisión experta	Evaluación experta en HCI y diseño pedagógico.	Coherencia del prompt, de los escenarios y de la arquitectura.	Revisión del repositorio, prompt, PoC y documento metodológico.	Antes o después del piloto.

6. Protocolos de interacción

Se proponen tres escenarios. Los dos primeros cumplen la recomendación mínima de la consigna; el tercero evalúa la evolución hacia memoria persistente.

Estructura temporal de una sesión piloto



Duración estimada: 60 minutos. Los logs se capturan durante toda la interacción con el agente.

Figura 5. Estructura temporal de una sesión piloto de aproximadamente 60 minutos.

Escenario 1: filtrado de números pares

Contexto	El participante debe resolver un ejercicio básico de listas, funciones y condicionales en Python. El agente debe guiar sin entregar la solución completa.
Tareas específicas	Leer el enunciado, iniciar interacción con el agente, proponer una estrategia, escribir código parcial y pedir ayuda ante dudas o errores.
Resultado esperado	El participante construye una función que recibe una lista y devuelve solo los números pares; el agente mantiene ayuda socrática y registra conceptos como listas, funciones, ciclos,

	condicionales y operador módulo.
Duración estimada	15-20 minutos

Escenario 2: secuencias numéricas rotativas

Contexto	El participante trabaja un problema que requiere manipulación de listas, índices, pop/append o slicing. Es un escenario útil para observar atasco y escalamiento de ayuda.
Tareas específicas	Pedir al agente orientación para crear la secuencia, intentar una rotación, depurar errores y continuar hasta obtener varias rotaciones.
Resultado esperado	El agente aumenta el nivel de andamiaje cuando aparece sticky count, distingue entre razonamiento y dudas declarativas, y evita entregar el código completo.
Duración estimada	15-20 minutos

Escenario 3: retorno con memoria persistente

Contexto	El participante inicia una nueva sesión después de haber interactuado previamente con el agente. Este escenario valida la recuperación del perfil del estudiante.
Tareas específicas	Iniciar sesión con usuario y contraseña, revisar el Perfil del Estudiante, resolver un nuevo ejercicio y observar si el agente ajusta la ayuda según conceptos previos.
Resultado esperado	El sistema recupera conversaciones anteriores, conceptos demostrados y nivel de apoyo histórico; el perfil muestra estados como Dominado, En progreso y Requiere apoyo.
Duración estimada	10-15 minutos

7. Diseño técnico de la persistencia y memoria

Esta sección resume la evolución técnica propuesta para convertir la PoC en un agente tutor con memoria persistente sin modificar el motor principal de andamiaje.

7.1 Entidades propuestas

Entidad	Campos principales	Relación	Finalidad
User	id, username, password_hash, created_at, last_login	1:N con Session	Autenticación y recuperación de historial.
Session	id, user_id, started_at, ended_at, mode, prompt_version	1:N con ChatEvent	Agrupar interacción por sesión.

ChatEvent	id, session_id, event_type, content, timestamp, response_time_ms	N:1 con Session	Persistir prompts, respuestas, errores y copias.
ScaffoldMetadata	event_id, scaffold_level, sticky_count, friction_ceiling	1:1 con ChatEvent	Persistir decisiones pedagógicas del motor.
StudentConcept	user_id, concept, state, evidence_count, last_observed_at	N:1 con User	Modelo persistente del estudiante.
StudentMetric	user_id, total_sessions, total_prompts, avg_response_time, last_scaffold_level	N:1 con User	Métricas acumuladas para seguimiento.

7.2 Endpoints REST propuestos

Endpoint	Método	Descripción
/api/auth/register	POST	Crear usuario con contraseña hasheada.
/api/auth/login	POST	Autenticar usuario y abrir sesión.
/api/sessions	GET	Listar sesiones del usuario autenticado.
/api/sessions/{id}	GET	Recuperar conversación completa de una sesión.
/api/chat	POST	Enviar prompt al agente y registrar evento + metadatos.
/api/profile	GET	Obtener perfil del estudiante con conceptos y estados.
/api/profile/recompute	POST	Recalcular perfil desde eventos históricos.
/api/import/json-logs	POST	Migrar logs JSON actuales hacia base de datos.

7.3 Estrategia de migración de JSON a base de datos

1. Leer todos los archivos JSON existentes y validar que contengan metadata de sesión.
2. Normalizar participantId para evitar duplicados por variantes como P03, 03 o 3.
3. Crear usuarios de migración con username derivado del participantId si aún no existe autenticación real.
4. Crear una sesión por cada sessionId original.
5. Insertar cada evento como ChatEvent conservando timestamp, eventType, prompt, response, error y responseTimeMs.
6. Extraer scaffoldLevel, stickyCount, conceptosDemostrados y frictionCeiling hacia ScaffoldMetadata cuando existan.

- 7. Recalcular StudentConcept por usuario usando reglas de evidencia observada.
- 8. Guardar un reporte de migración con conteo de sesiones, eventos importados y errores.

8. Repositorio, video y documentación

El repositorio debe reflejar el avance técnico y metodológico de la PoC. La estructura recomendada separa código, prompts y documentos para que el profesor pueda ejecutar, revisar y auditar el proyecto sin depender de explicaciones externas.

Estructura recomendada: docs/ para documentos PDF, prompts/ para system prompts y README, src/ para la PoC, README.md raíz con instrucciones de ejecución y enlace al video no listado.

Elemento de entrega	Ubicación sugerida	Estado
Documento de avance en PDF	docs/ Entregable2_Avance_Agente_Ha nsellSolis.pdf	Generado a partir de este documento.
PoC funcional	src/	Implementada con chat, sesiones, logs y motor de andamiaje.
Prompts	prompts/	README actualizado y prompt v2 documentado.
Documentos complementarios	docs/evaluacion/ y docs/propuesta_v2/	Informe integrado, artículo técnico y propuesta v2.
Video de demostración	README.md raíz	Pendiente de pegar enlace no listado.
API keys	.env local, excluido por .gitignore	No subir credenciales al repositorio.

9. Checklist de cumplimiento de la rúbrica

Criterio	Peso	Evidencia en este avance	Estado
Prueba de Concepto	30%	Arquitectura actual, chat LLM, sesiones, logs, niveles, sticky count y detección de conceptos.	Cumple / mejorar video.
Diseño metodológico	25%	Tipo de estudio, participantes, procedimiento y justificación ética.	Cumple.
Métricas e instrumentos	20%	SUS, NASA-TLX, logs, fidelidad del prompt, entrevista/focus group y revisión experta.	Cumple.
Protocolos de	10%	Tres escenarios con	Cumple.

interacción		contexto, tareas, resultado esperado y duración.	
Video de demostración	5%	Debe mostrar interacción real y al menos un escenario.	Pendiente enlace.
Documentación y repositorio	10%	README raíz, README de prompts, docs y estructura src.	Cumple si se actualiza repo.

10. Conclusión

El proyecto muestra un avance sustancial respecto al Entregable 1. Ya no se presenta únicamente una propuesta conceptual, sino una PoC funcional con integración entre interfaz, motor de andamiaje, LLM y registro de eventos. La principal decisión de diseño fue consolidar primero el núcleo socrático y su instrumentación antes de escalar hacia embodiment completo o memoria persistente. Esta decisión es coherente con el objetivo de evitar que el agente sustituya el razonamiento del estudiante y con la necesidad de evaluar el sistema mediante evidencia reproducible.

La siguiente etapa es técnica y metodológicamente clara: implementar autenticación, persistencia en base de datos, migración de logs, perfil del estudiante y visualización del historial. Estas mejoras no reemplazan la arquitectura actual de tres capas ni el motor de conversación, sino que los fortalecen para convertir el prototipo en un agente tutor con memoria persistente y evaluación longitudinal.

Referencias

- Brooke, J. (1996). SUS: A quick and dirty usability scale. En P. W. Jordan et al. (Eds.), Usability Evaluation in Industry. Taylor & Francis.
- Hart, S. G., & Staveland, L. E. (1988). Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. *Advances in Psychology*, 52, 139-183.
- Liffiton, M., Sheese, B. E., Savelka, J., & Denny, P. (2023). CodeHelp: Using LLMs with Guardrails for Scalable Support in Programming Classes. Koli Calling.
- Sauro, J., & Lewis, J. R. (2016). Quantifying the User Experience: Practical Statistics for User Research. Morgan Kaufmann.
- Wood, D., Bruner, J. S., & Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2), 89-100.
- Solís Ramírez, H. (2026). Entregable 1: Propuesta de Agente e Investigación. PF-3311 Agentes Virtuales Inteligentes, Universidad de Costa Rica.
- Solís Ramírez, H. (2026). Informe integrado de evaluación del agente conversacional socrático. Documento complementario de investigación.